

Modular Answer Sheet

Google Photorealistic Tiles • Site Boundary • IFC Model • Saved Views • CSV-linked Properties

Student workflow: Paste Section 1 into a blank Sandcastle editor. Run it. Then append one section at a time in numerical order, running the application after every addition. Do not replace earlier sections.

Before you start

- Open Cesium Sandcastle and work only in the JavaScript panel.
- Replace **YOUR_CESIUM_ION_ACCESS_TOKEN** with the workshop token supplied by the instructor.
- The proposed-building asset ID is 4671415. The site boundary and CSV source are already configured.
- If a section fails, check the browser console before adding the next section.
- Token:
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiIyYTRmMDgzNi0xYTVjLTQzYmUtYWVkbmNy1hOTE5OGFhYTM5O",
WYiLCJpZCI6ImJkwOTA3LCJpYXQiOiJlE3NDM3NzAxOTI9.0h5bZmhgMCUAcCmzqK74T8T05D7e_u6_a8DUbK7nGzg"

Section 1 • Create the viewer and project context

Outcome: Creates the Cesium viewer, loads Google Photorealistic 3D Tiles, and draws the red site boundary.

```
// =====
// 1. Access token, performance settings and viewer
// =====
Cesium.Ion.defaultAccessToken = "YOUR_CESIUM_ION_ACCESS_TOKEN";
console.log("Cesium ion access token set");

Cesium.RequestScheduler.requestsByServer["tile.googleapis.com:443"] = 18;
console.log("Google tile request concurrency configured");

let viewer;

try {
  viewer = new Cesium.Viewer("cesiumContainer", {
    globe: false,
    baseLayerPicker: false,
    timeline: false,
    animation: false,
    geocoder: Cesium.IonGeocoderProviderType.GOOGLE
  });
  console.log("Viewer created with Google geocoder");
} catch (err) {
  console.error("Viewer creation failed:", err);
}

// =====
// 2. Google Photorealistic 3D Tiles
// =====
(async function () {
  try {
    console.log("Loading Google Photorealistic 3D Tiles...");
    const googleTileset =
      await Cesium.createGooglePhotorealistic3DTilesset({
        onlyUsingWithGoogleGeocoder: true
      });
    viewer.scene.primitives.add(googleTileset);
    console.log("Google tiles loaded");
  } catch (err) {
    console.error("Google tiles failed:", err);
  }
})();

// =====
// 3. Site boundary
// =====
try {
  const boundaryPositions = Cesium.Cartesian3.fromDegreesArray([
    -1.1026274713583746, 50.795421191436276,
    -1.1025838615994417, 50.79499852088367,
    -1.1018109997604597, 50.79503221215484,
    -1.1018570322839594, 50.795457945216896,
    -1.1026274713583746, 50.795421191436276
  ]);

  viewer.entities.add({
    name: "Site Boundary",
    polyline: {
      positions: boundaryPositions,
      width: 6,
      material: Cesium.Color.RED,
      clampToGround: true
    }
  });
  console.log("Site boundary added");
} catch (err) {
  console.error("Failed to add site boundary:", err);
}
```

Checkpoint: The 3D basemap should load and a thick red boundary should appear. It may take a moment for nearby tiles to sharpen.

Section 2 • Load the proposed IFC building

Outcome: Adds the Cesium ion asset, stores it for later controls, and zooms to it.

```
// =====  
// 4. Proposed building  
// =====  
let proposedTileset = null;  
  
(async function () {  
  try {  
    console.log("Loading Proposed Building (Asset 4671415)...");  
    proposedTileset =  
      await Cesium.Cesium3DTileset.fromIonAssetId(4671415);  
  
    viewer.scene.primitives.add(proposedTileset);  
  
    if (proposedTileset.readyPromise) {  
      await proposedTileset.readyPromise;  
    }  
  
    console.log("Proposed Building loaded");  
    await viewer.zoomTo(proposedTileset);  
  } catch (err) {  
    console.error("Proposed Building failed:", err);  
  }  
})();
```

Checkpoint: The proposed building should appear and the camera should move to it automatically.

Section 3 • Add saved camera views

Outcome: Creates Top, Front, Back and Side buttons that move relative to the loaded building.

```
// =====
// 5. Views widget and camera helper
// =====
const viewsWidget = document.createElement("div");
viewsWidget.innerHTML = `
  <div style="position:absolute;bottom:10px;left:10px;
    background:rgba(42,42,42,0.9);color:white;padding:8px;
    border-radius:4px;font-family:sans-serif;font-size:13px;">
    <strong>Views</strong>
    <div style="margin-top:6px;display:grid;
      grid-template-columns:1fr 1fr;gap:4px;">
      <button id="viewTop">Top</button>
      <button id="viewFront">Front</button>
      <button id="viewBack">Back</button>
      <button id="viewSide">Side</button>
    </div>
  </div>
`;
viewer.container.appendChild(viewsWidget);

viewsWidget.querySelectorAll("button").forEach(function (button) {
  button.style.cursor = "pointer";
  button.style.background = "#3a3a3a";
  button.style.color = "white";
  button.style.border = "1px solid #666";
  button.style.borderRadius = "3px";
});

function flyToBuildingView(headingDeg, pitchDeg, rangeMultiplier) {
  rangeMultiplier = rangeMultiplier || 2.5;

  if (!proposedTileset || !proposedTileset.boundingSphere) {
    console.warn("Building not ready for camera view");
    return;
  }

  const sphere = proposedTileset.boundingSphere;
  viewer.camera.flyToBoundingSphere(sphere, {
    offset: new Cesium.HeadingPitchRange(
      Cesium.Math.toRadians(headingDeg),
      Cesium.Math.toRadians(pitchDeg),
      sphere.radius * rangeMultiplier
    ),
    duration: 3.3
  });
}

viewsWidget.querySelector("#viewTop").onclick = function () {
  flyToBuildingView(0, -90, 2.2);
};
viewsWidget.querySelector("#viewFront").onclick = function () {
  flyToBuildingView(0, -15, 2.5);
};
viewsWidget.querySelector("#viewBack").onclick = function () {
  flyToBuildingView(180, -15, 2.5);
};
viewsWidget.querySelector("#viewSide").onclick = function () {
  flyToBuildingView(90, -15, 2.5);
};

console.log("View buttons created and connected");
```

Checkpoint: Four buttons should appear in the bottom-left corner. Each should move the camera without using fixed world coordinates.

Section 4 • Add the Layers panel

Outcome: Creates a top-right panel that can show or hide the proposed building.

```
// =====
// 6. Layers widget and visibility toggle
// =====
const layersWidget = document.createElement("div");
layersWidget.innerHTML = `
  <div id="featureWidget" style="position:absolute;top:10px;right:10px;
    background:rgba(42,42,42,0.9);color:white;padding:10px;
    border-radius:4px;font-family:sans-serif;font-size:13px;
    min-width:200px;">
    <div style="display:flex;justify-content:space-between;
      align-items:center;">
      <strong>Layers</strong>
      <span id="closeWidget" style="cursor:pointer;">X</span>
    </div>
    <hr style="border:0;border-top:1px solid #666;margin:6px 0;" />
    <label>
      <input type="checkbox" id="toggleProposed" checked />
      Proposed Building
    </label>
  </div>
`;
viewer.container.appendChild(layersWidget);

layersWidget.querySelector("#closeWidget").onclick = function () {
  layersWidget.style.display = "none";
};

layersWidget.querySelector("#toggleProposed")
  .addEventListener("change", function (event) {
    if (proposedTileset) {
      proposedTileset.show = event.target.checked;
      console.log("Proposed Building visibility:", event.target.checked);
    } else {
      console.warn("Building is still loading");
    }
  });

console.log("Layers widget created");
```

Checkpoint: The Layers panel should appear in the top-right. Unticking the checkbox should hide only the proposed building.

Section 5 • Load the IFC property CSV

Outcome: Downloads the CSV and builds a lookup object keyed by IFC GlobalId.

```
// =====
// 7. Load CSV and build an IFC GlobalId lookup
// =====
const IFC_CSV_URL =
  "https://raw.githubusercontent.com/JonaSchubert/Digital-Twin-DIY/main/DTcsv.csv";

const ifcCsvLookup = Object.create(null);
let ifcCsvReady = false;

(async function () {
  try {
    console.log("Loading IFC CSV from GitHub...");
    const response = await fetch(IFC_CSV_URL);

    if (!response.ok) {
      throw new Error("CSV fetch failed: " + response.status);
    }

    const csvText = (await response.text()).replace(/^\\uFEFF/, "");
    const lines = csvText
      .split(/\\r?\\n/)
      .filter(function (line) { return line.trim().length > 0; });
    const headers = lines[0].split(",").map(function (header) {
      return header.trim();
    });

    let count = 0;
    for (let i = 1; i < lines.length; i++) {
      const values = lines[i].split(",").map(function (value) {
        return value.trim();
      });
      const row = {};

      headers.forEach(function (header, index) {
        row[header] = values[index] || "";
      });

      const globalId = row.globalId || row.GlobalId;
      if (globalId) {
        ifcCsvLookup[globalId] = row;
        count++;
      }
    }

    ifcCsvReady = true;
    console.log("IFC CSV loaded:", count, "records");
  } catch (err) {
    console.error("Failed to load IFC CSV:", err);
  }
})();
```

Checkpoint: The console should report that the IFC CSV loaded and show the number of indexed records.

Section 6 • Click, highlight and display linked properties

Outcome: Highlights the selected IFC element and displays its matching CSV row in the Cesium information panel.

```
// =====
// 8. IFC feature selection and CSV property display
// =====
let lastHighlightedFeature = null;
let lastFeatureOriginalColor = null;

const csvInfoEntity = new Cesium.Entity({
  name: "IFC + CSV Properties"
});
viewer.entities.add(csvInfoEntity);

const handler = new Cesium.ScreenSpaceEventHandler(viewer.scene.canvas);

handler.setInputAction(function (movement) {
  try {
    const picked = viewer.scene.pick(movement.position);

    if (lastHighlightedFeature) {
      lastHighlightedFeature.color = lastFeatureOriginalColor;
      lastHighlightedFeature = null;
      lastFeatureOriginalColor = null;
    }

    if (
      Cesium.defined(picked) &&
      picked instanceof Cesium.Cesium3DTileFeature &&
      picked.tileset === proposedTileset
    ) {
      lastHighlightedFeature = picked;
      lastFeatureOriginalColor = Cesium.Color.clone(picked.color);
      picked.color = Cesium.Color.YELLOW.withAlpha(0.85);

      const globalId =
        picked.getProperty("globalId") ||
        picked.getProperty("GlobalId");

      console.log("Resolved GlobalId:", globalId);

      if (!ifcCsvReady) {
        console.warn("CSV is not loaded yet");
        return;
      }

      const csvData = ifcCsvLookup[globalId];
      if (!csvData) {
        console.warn("No CSV match for this GlobalId");
        return;
      }

      let html = "<table style='width:100%;font-size:12px;'>";
      Object.keys(csvData).forEach(function (key) {
        html +=
          "<tr><td><strong>" + key + "</strong></td>" +
          "<td>" + csvData[key] + "</td></tr>";
      });
      html += "</table>";

      csvInfoEntity.description = html;
      viewer.selectedEntity = csvInfoEntity;
    }
  } catch (err) {
    console.warn("Picking error:", err);
  }
}, Cesium.ScreenSpaceEventType.LEFT_CLICK);

console.log("IFC feature picking enabled");
```

Checkpoint: Click an IFC component. It should turn yellow. If its GlobalId exists in the CSV, its linked properties should open in the information panel.

Troubleshooting checklist

The viewer does not load: Check that the access-token placeholder has been replaced and inspect the first console error.

The building does not appear: Confirm that asset 4671415 is accessible to the Cesium ion account associated with the token.

The camera buttons do nothing: Wait for the building to finish loading, then click the button again.

The CSV does not load: Open the console and check the HTTP status shown by the CSV fetch error.

A component highlights but shows no table: Check that the selected feature exposes globalId or GlobalId and that the same value exists in the CSV.

Code shows < or =>: Paste from this answer sheet rather than from HTML-rendered chat text. The code here uses normal JavaScript characters.